SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL – ASSIGNMENT 3

Course Code:	CSA17	**Course Title:**	Artificial Intelligence
CO Assessed:	CO3	**Assessment:**	Assignment 3 – Pair Work: Knowledge and Reasoning
Weightage:	2% of CIA	**Duration / Marks:**	50 marks
CO3 Statement:	Analyze knowledge representation and reasoning using propositional logic, FOL, inference, resolution, chaining methods, Knowledge representation to perform logical reasoning for drawing valid conclusions and decision making. (BL4)		
Student A / B Names:	Tejhash P	**Reg. Nos.:**	192512495
Section / Batch:	1	**Date:**	

Code of Conduct

I Tejhash P, 192512495 _____ *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the Student: _____

Instructions

- Read each task carefully before starting. This is a take-home assignment for a pair of students (Student A and Student B), completed over one week.
- Each task is a problem drawn from the prescribed textbook (Russell and Norvig, AIMA). Select the technique that fits, apply it step by step, and support every decision with evidence.
- The pair must not divide the assignment into two independent halves; each student must be able to run and trace the other student's component, and a contribution record must be submitted.
- Tasks are drawn from Logical Agents, Propositional Logic, Resolution, First-Order Logic, Knowledge Engineering, Inference in FOL, Unification, Forward and Backward Chaining.
- Every implemented task must be supported by: Source Code → Test Input → Expected Behaviour → Actual Output → Performance Metrics → Interpretation. Where a task says Predict, write the prediction before running the experiment; use fixed random seeds. Programming language: Python 3 (Jupyter / Colab).
- Show CNF conversions, proof trees, unifiers, metric tables and traces, and state the final answer clearly.
- Marks will be awarded using the Assignment rubric: conceptual understanding, application of concepts, analytical thinking, organization & presentation, and completeness & accuracy.
- Upload your work in GCR and Github.

Reference Books Note

- T1 – Stuart Russell and Peter Norvig, Artificial Intelligence: A Modern Approach, 3rd Edition, PHI / Pearson Education (AIMA).
- Chapters used: Chapter 7 (wumpus world Section 7.2, model checking and Figure 7.10, resolution and Figure 7.12, DPLL Section 7.6, hybrid wumpus agent Section 7.7, Exercises 7.12 and 7.20); Chapter 8 (first-order logic, knowledge engineering Section 8.4, kinship domain Section 8.3.2); Chapter 9 (unification and Figure 9.1, forward chaining Figure 9.3, backward chaining Figure 9.6, resolution Section 9.5, the crime problem of Section 9.3.1, Exercises 9.4 and 9.19).
- Figure, table, section and exercise numbers in this assignment refer to the editions used for the problems: AIMA 3rd edition (T1).

Section / Topic	Focus	Task Nos.

1. Propositional Logic	Knowledge-based agents, wumpus-world KB, model checking, resolution, DPLL	**Tasks 1–2**
2. First-Order Logic	Knowledge engineering, unification and lifting, FOL resolution	**Tasks 3–5**
3. Chaining and Design Challenge	Forward and backward chaining; hybrid wumpus agent (Day 4)	**Tasks 6–7**

Rubrics for Assignment

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Conceptual Understanding	**25%**	Demonstrates complete and accurate understanding of concepts with strong clarity	Shows good understanding with minor gaps	Partial understanding with some errors	Lacks basic understanding of concepts
Application of Concepts	**25%**	Applies concepts correctly to problems with logical and appropriate solutions	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply concepts correctly
Analytical Thinking	**20%**	Provides in-depth analysis with strong reasoning and justification	Shows logical analysis with some justification	Basic analysis with weak reasoning	No meaningful analysis provided
Organization & Presentation	**15%**	Well-structured, neat, and logically organized with clear flow	Generally organized with minor issues	Some structure but lacks clarity	Poorly organized and difficult to follow
Completeness & Accuracy	**15%**	All questions attempted with accurate and error-free answers	Most answers correct with minor mistakes	Some answers incomplete or incorrect	Many answers missing or incorrect

Task / Criteria	Conceptual Understanding (25%)	Application of Concepts (25%)	Analytical Thinking (20%)	Organization & Presentation (15%)	Completeness & Accuracy (15%)	Sub. Total (Max)	Total %
Task 1 (6)						__ / 6	
Task 2 (7)						__ / 7	
Task 3 (7)						__ / 7	
Task 4 (7)						__ / 7	
Task 5 (6)						__ / 6	
Task 6 (7)						__ / 7	
Task 7 (10)						__ / 10	
Total (50)	__ / 12.5	__ / 12.5	__ / 10	__ / 7.5	__ / 7.5	__ / 50	

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____

Date: _____	Date: _____	Date: _____

Task 1: Analyze Logical Agents: Knowledge Bases, Entailment and Reasoning Patterns in Propositional Logic (6 Marks)

CO3 Topic Mapping: Topic 1 – Knowledge-Based Agents and Logical Agents; Topic 2 – Propositional Logic; Topic 3 – Reasoning Patterns in Propositional Logic

Knowledge-based agent: design the wumpus-world agent of AIMA Section 7.2 as a knowledge-based (logical) agent by writing its knowledge base, the TELL and ASK operations and the agent loop (percept → TELL → ASK → action) that is used throughout this assignment. Each square other than [1,1] may contain a pit with probability 0.2, there is one wumpus, and the agent perceives breeze, stench and glitter.

Part A – Knowledge Base and Model Checking in the Wumpus World

Shared Group Work: Together encode the rules of an n × n wumpus world in propositional logic (pit and wumpus symbols, breeze and stench symbols for every square, and the rules relating percepts to neighbouring hazards). Implement a truth-table entailment checker (TT-ENTAILS, AIMA Figure 7.10) and reproduce the book's example: the KB R1: $\neg P_{1,1}$; R2: $B_{1,1} \Leftrightarrow (P_{1,2} \vee P_{2,1})$; R3: $B_{2,1} \Leftrightarrow (P_{1,1} \vee P_{2,2} \vee P_{3,1})$ with the percepts $\neg B_{1,1}$ and $B_{2,1}$ has $2^7 = 128$ candidate models, of which KB is true in only three. Query it with $\alpha 1 = \neg P_{1,2}$ and $\alpha 2 = \neg P_{2,2}$.

Student A – Design and Mathematical Analysis

(a) Count the proposition symbols and sentences as a function of n, and compute the number of models 2^m for n = 4 (32 pit and breeze symbols give $2^{32} = 4{,}294{,}967{,}296$)

(b) Identify the minimal set of symbols relevant to one query

(c) Verify soundness and completeness of model checking on three test KBs (an entailed query, a non-entailed query and an inconsistent KB)

Student B – Experimental Validation

(a) Instrument the checker to record models enumerated, KB evaluations and time

(b) Run it for n = 2, 3, 4 (record the n at which the full symbol set becomes infeasible) and again with relevant-symbol restriction

(c) Include one entailed query ($\alpha 1$), one query whose negation is entailed and one query for which neither is entailed ($\alpha 2$)

Part B – Reasoning Patterns

Shared Group Work: Using the same wumpus-world KB, derive the safety of at least two squares by applying inference rules (Modus Ponens, And-Elimination, Unit Resolution) instead of model checking, following the derivation of AIMA Section 7.5.

Student A – Experimental Investigation: Implement the inference-rule derivation, record the number of inference steps and compare with the model-checking cost for the same query.

Student B – Soundness Reasoning: Prove the soundness of each rule used with a truth table and construct a KB and a query for which "not derived" differs from "false".

Cross-Understanding Requirement: Each student must be able to run and trace the other student's component.

Evidence Expected: KB listing, symbol-count derivation, checker code and outputs for the three query types, inference-rule derivations with step counts, soundness tables and a comparison of the two reasoning approaches.

Task 2: Analyze Inference in Propositional Logic: Model Checking versus Resolution (7 Marks)

CO3 Topic Mapping: Topic 4 – Resolution in Propositional Logic

Shared Group Work: Implement a truth-table model checker, a propositional resolution prover (CNF conversion followed by resolution refutation, AIMA Figure 7.12) and the DPLL algorithm (Section 7.6). Use two knowledge bases from the book: the wumpus-world KB of Task 1, and the six sentences of AIMA Exercise 7.20: S1: $A \Leftrightarrow (B \vee E)$; S2: $E \Rightarrow D$; S3: $(C \wedge F) \Rightarrow \neg B$; S4: $E \Rightarrow B$; S5: $B \Rightarrow F$; S6: $B \Rightarrow C$.

The group must design test knowledge bases that make each method look strong or weak. Do not use a single sample KB.

Student A – Resolution Investigation

(a) Convert S1–S6 to CNF by hand and construct a resolution proof of ¬A ∧ ¬B (AIMA Exercise 7.12)
(b) Record clauses generated, resolvents attempted, tautologies discarded, subsumed clauses removed and time as the KB grows (implication chains of length 10, 20, 40, 80)
(c) Test a query that is not entailed and determine from the clause count when the process terminates

Student B – Model-Checking and DPLL Comparative Analysis

(a) Record models enumerated and time for the same KBs and queries; trace DPLL on the clauses of Exercise 7.20 and count the models of the KB over six symbols
(b) Identify a KB family on which model checking is faster than resolution and a family on which it is far slower, and determine which structural property (for example clause length or number of symbols) causes the difference

Joint Design Question

If you were designing a reasoning component whose KB size and query types are not known in advance, what information would you want before deciding between these methods? Support your answer with your own measurements.

Evidence Expected: Manual CNF conversion, proof tree, three implementations, clause and model counts across KB sizes, a non-entailed-query test and the joint design reasoning.

Task 3: Analyze Knowledge Representation in First-Order Logic: Knowledge Engineering (7 Marks)

CO3 Topic Mapping: Topic 5 – First-Order Logic (FOL); Topic 6 – Knowledge Engineering using First-Order Logic

Shared Group Work: Follow the knowledge-engineering process of AIMA Section 8.4 (identify the task, assemble relevant knowledge, decide on a vocabulary, encode general knowledge, encode the problem instance, pose queries, debug) for the crime problem of AIMA Section 9.3.1: "The law says that it is a crime for an American to sell weapons to hostile nations. The country Nono, an enemy of America, has some missiles, and all of its missiles were sold to it by Colonel West, who is American." Then encode the kinship domain of AIMA Section 8.3.2 (Parent, Sibling, Grandparent, Female) with at least 12 axioms.

Student A – Vocabulary and Axioms

(a) Design the FOL vocabulary and the axioms for both domains, showing the choice between predicates and functions (for example, Mother(x) or Parent(m, x) ∧ Female(m))
(b) Derive the growth of the number of ground sentences with the number of constants k for the crime KB (the rule with three variables yields k^3 ground instances)

Student B – Query Suite and Defect Hunt

(a) Design a query suite with expected and actual results (for example, Criminal(West), Hostile(Nono), Sibling(x, y), Grandparent(z, w))
(b) Find a deliberate defect (a missing axiom such as "missiles are weapons") and show which query fails and how the defect is located

Evidence Expected: Vocabulary and axioms, growth derivation, query suite with expected and actual results, defect demonstration and the knowledge-engineering log.

Task 4: Analyze Inference in First-Order Logic: Unification, Lifting and Propositional versus First-Order Inference (7 Marks)

CO3 Topic Mapping: Topic 7 – Inference in First-Order Logic; Topic 8 – Propositional vs. First-Order Inference and Unification

Shared Group Work: Implement the UNIFY algorithm of AIMA Figure 9.1 with standardizing apart and the occurs check. Test it on the book's examples: Knows(John, x) against Knows(John, Jane), Knows(y, Bill), Knows(y, Mother(y)) and Knows(x, Elizabeth), and on the pairs of AIMA Exercise 9.4: P(A, B, B) with P(x, y, z); Q(y, G(A, B)) with Q(G(x, x), y); Older(Father(y), y) with Older(Father(x), John); Knows(Father(y), y) with Knows(x, x).

Student A – Hand Derivations and Propositionalisation

(a) Derive the most general unifier of each pair by hand, or show that none exists, and show which pairs fail without standardizing apart

(b) Propositionalise the crime KB over the constants West, Nono, M1 and America and compute the ground clause counts (64 + 4 + 4 + 4 = 76 against 4 lifted rules); extend to k = 8, 16, 32 constants

Student B – Lifted versus Ground Inference

(a) Implement ground (propositionalised) forward chaining and lifted forward chaining for the crime KB; record inferences and time as the number of irrelevant constants grows

(b) Remove the occurs check and find the test pair that now gives a wrong answer

Evidence Expected: Unifier code and tests, hand derivations, ground-clause growth tables, a lifted-versus-ground comparison and the occurs-check demonstration.

Task 5: Analyze Resolution in First-Order Logic: Drawing Valid Conclusions (6 Marks)

CO3 Topic Mapping: Topic 4 – Resolution in Propositional Logic; Topic 5 – First-Order Logic (FOL); Topic 9 – Forward Chaining, Backward Chaining and Resolution

Shared Group Work: Implement a first-order resolution prover. Use the two problems of AIMA Section 9.5: (1) "Everyone who loves all animals is loved by someone", converted to CNF with Skolemisation; (2) "Jack owns a dog. Every dog owner is an animal lover. No animal lover kills an animal. Either Jack or Curiosity killed the cat, who is named Tuna. Did Curiosity kill the cat?"

Student A – Conversion and Proof Design

(a) Convert both problems to CNF by hand, deriving the Skolem function or constant with its arguments at every existential quantifier

(b) Construct the resolution proof tree for the Curiosity question, with the unifier at every step

Student B – Strategy Comparison

(a) Run the prover with breadth-first, unit-preference and set-of-support strategies (AIMA Section 9.5.6); record clauses generated and steps

(b) Construct a small example showing that Skolemisation preserves satisfiability but not logical equivalence

(c) Query a sentence that is not entailed and record what the prover does

Evidence Expected: Conversion steps, proof tree, prover code, strategy comparison, behaviour on the non-entailed query and the Skolemisation example.

Task 6: Analyze Forward Chaining and Backward Chaining (7 Marks)

CO3 Topic Mapping: Topic 9 – Forward Chaining, Backward Chaining and Resolution

Shared Group Work: Implement forward chaining (AIMA Figure 9.3) and backward chaining (Figure 9.6) over definite clauses. Validate them on the crime KB: forward chaining must reach Criminal(West) in two iterations (the first adds Sells(West, M1, Nono), Weapon(M1) and Hostile(Nono)). Then use the ancestor KB of AIMA Exercise 9.19: Ancestor(Mother(x), x) and Ancestor(x, y) $\wedge$ Ancestor(y, z) $\Rightarrow$ Ancestor(x, z), and the queries Ancestor(Mother(y), John), Ancestor(Mother(Mother(y)), John) and Ancestor(Mother(Father(John)), Mother(Mother(John))).

Student A – Backward-Chaining Investigation

(a) Trace the backward-chaining proof of Criminal(West) as an AND/OR tree and record goals expanded, unifications and maximum depth

(b) Demonstrate the non-termination of the last ancestor query under forward chaining and design a remedy

Student B – Forward-Chaining Investigation

(a) Record iterations, rules fired and facts derived for forward chaining at fact-base sizes of 100, 1,000 and 10,000 irrelevant facts

(b) Record goals expanded for backward chaining on the same sizes and the repeated subgoals

Verification Questions

(a) Student B must analyze the choice of inference direction by comparing forward-chaining and backward-chaining step counts on this KB

(b) Student A must evaluate backward chaining by adding 1,000 irrelevant facts to the KB and comparing the step counts

Evidence Expected: Two working engines, metric tables for the three fact sizes, the non-termination demonstration, the remedy, and correct responses to the verification questions.

DAY-4 DESIGN CHALLENGE

Task 7: Perform Logical Reasoning for Decision Making: Design a Knowledge-Based Reasoning Component from First Principles (10 Marks)

CO3 Topic Mapping: Topics 1–9 integrated (knowledge representation, inference, resolution and chaining methods)

Problem Context – The Hybrid Wumpus Agent (AIMA Section 7.7)

In a 4 × 4 wumpus world the agent must decide, from its percepts so far, which squares are provably safe, which are possibly dangerous, and what to do next (move to a safe unvisited square, shoot, grab the gold, or return). Each non-start square may hold a pit with probability 0.2.

This problem must be solved collaboratively. Neither student may work independently and merge the answers later.

Objective: use knowledge representation and logical reasoning to draw valid conclusions (is the square OK, and why?) and to support the decision on the next action.

Stage 1 – Independent Design Proposal – 1 Mark

Before discussing with each other, Student A and Student B must each independently propose: the representation (propositional with time-indexed fluents, or FOL), the vocabulary, the inference method (model checking, resolution, DPLL, forward or backward chaining) and how the agent chooses among safe squares.

Stage 2 – Compare the Two Designs – 1 Mark

Identify where the designs agree and differ, their strengths and weaknesses, and which ideas will be retained.

Stage 3 – Joint Final Design – 3 Marks

Produce the final design: representation, vocabulary, inference method, the way the KB is kept small as time advances, the explanation of each safety conclusion and complete pseudocode of the agent loop.

Stage 4 – Divide the Validation Responsibilities – 3 Marks

(a) Student A – Theoretical Validation: soundness and completeness of the chosen inference, the growth of a query's cost with the number of squares and time steps, and the effect of restricting the KB to definite clauses.

(b) Student B – Experimental Validation: at least 30 random worlds on 4 × 4, 6 × 6 and 8 × 8 grids, metrics (fraction of worlds solved, unnecessary risks taken, inference steps per move, KB size) and one world in which logical reasoning alone cannot decide a safe move.

Stage 5 – Cross-Defence – 1 Mark

Student A analyzes the experimental evidence supporting the group's design. Student B analyzes whether the theoretical complexity result applies to the group's chosen inference.

Stage 6 – Joint Evaluation – 1 Mark

Why is the problem suitable for a logical agent? What are the limits of the representation (uncertainty, incomplete data)? What should an AI designer check before choosing a knowledge-based approach?

CSA17 Artificial Intelligence — CO3 Assignment 3

Worked solutions and reproducible experiment guide

Students A/B: _____ / _____

Registration numbers: _____ / _____

Section / Date: _____ / _____

Academic integrity note: This is a worked study draft. Both students must run the code, verify traces, enter their own measured results and accurately document contributions before submitting. Timings are machine-dependent. Do not present unrun experiments as observations.

Task 1 — Knowledge-based Wumpus agent (6 marks)

Knowledge base and agent cycle

For each square `(i,j)`, introduce `Pij` (pit), `Wij` (Wumpus), `Bij` (breeze), `Sij` (stench), `Gij` (glitter), and `OKij` (safe). For orthogonally adjacent cells `N(i,j)`, assert:

• `Bij <=> OR(Puv for (u,v) in N(i,j))`.

• `Sij <=> OR(Wuv for (u,v) in N(i,j))`.

• `OKij <=> (~Pij AND ~Wij)`.

• `~P11 AND ~W11`; exactly one Wumpus (at least one `Wij`, and pairwise at-most-one).

• An observation is TELLed as a literal: e.g., no breeze at `(1,1)` means `~B11`.

The 0.2 pit probability is a *world-generation prior*, not a propositional logical fact. Logical entailment never turns probability 0.2 into a proof of safety. A simple agent repeats `percept = sense(); TELL(KB, percept at position/time); safe = {q: ASK(KB, OK(q))}; choose grab if glitter, otherwise safe unvisited movement, otherwise return or decide whether a risk is allowed`.

Book example and truth-table entailment

Seven symbols are `P11,P12,P21,P22,P31,B11,B21`, hence 128 assignments. The KB contains `~P11`, `B11 <=> (P12 OR P21)`, `B21 <=> (P11 OR P22 OR P31)`, `~B11`, and `B21`. Consequently `P11=P12=P21=B11=0`, `B21=1`, and `(P22,P31)` can be `(1,0),(0,1),(1,1)`: **three satisfying models**.

`KB |= ~P12` is **true**; `KB |= P12` is **false**; neither `P22` nor `~P22` is entailed. An inconsistent KB entails every query under classical logic (vacuous entailment), which must not be confused with a useful conclusion.

Symbol and sentence growth

For a pit+breeze-only `n*n` grid there are `2n²` symbols, `n²` breeze biconditionals and one initial no-pit assertion (plus observations). Thus `2^(2n²)` assignments: n=2: 256; n=3: 262,144; n=4: 4,294,967,296. Including Wumpus and stench gives `4n²` symbols before other fluents. Relevant-symbol restriction is essential. For the example query `~P12`, only `P12,P21,B11` and `~B11` plus the first biconditional are needed; `P11,P22,P31,B21` do not affect the conclusion. For `P22`, use the full local seven-symbol set.

Inference-rule derivation and soundness

From `~B11` and `B11 <=> (P12 OR P21)`, obtain `~P12 AND ~P21`; by **and-elimination**, conclude `~P12` and `~P21`. From the unit clauses `~P12 OR B11` and `~B11`, **unit resolution** gives `~P12`. `~P11` is known, so `(1,1),(1,2),(2,1)` are pit-free. They are *provably safe* only after independently proving the Wumpus absent at those cells.

Modus ponens is sound because `p AND (p=>q) AND ~q` is unsatisfiable. And-elimination is sound because `(p AND q) AND ~p` is unsatisfiable. Unit resolution is sound because `(p OR q) AND ~p AND ~q` is unsatisfiable. For every rule, the premises-true/conclusion-false truth-table row count is zero. **Not derived is not false**: with KB `{P OR Q}`, neither `P` nor `~P` follows.

Three checker tests

1. `{P, P=>Q} |= Q`: true. 2. `{P OR Q} |= P`: false. 3. `{P,~P} |= R`: true (inconsistent KB). The provided Python implementation prints model counts and elapsed times. **Do not exhaustively enumerate the full n=4 32-symbol space**; estimate and label the infeasible run

instead.

Task 2 — Model checking, resolution, DPLL (7 marks)

Hand CNF for AIMA Exercise 7.20

Given `A <=> (B OR E)`, `E=>D`, `(C AND F)=>~B`, `E=>B`, `B=>F`, `B=>C`, the CNF clauses are:

1. `(~A OR B OR E)`; 2. `(~B OR A)`; 3. `(~E OR A)`;

4. `(~E OR D)`; 5. `(~C OR ~F OR ~B)`;

6. `(~E OR B)`; 7. `(~B OR F)`; 8. `(~B OR C)`.

Resolution proof of `~B`: Resolve clause 8 `( ~B OR C)` with clause 5 `(~C OR ~F OR ~B)` on `C` to obtain `(~B OR ~F)`; resolve with clause 7 `(~B OR F)` on `F` to obtain `~B`. Then clause 6 `(~E OR B)` with `~B` yields `~E`. Clause 1 `(~A OR B OR E)` with `~B` yields `(~A OR E)`; with `~E` yields `~A`. Thus `KB |= ~A AND ~B`.

For refutation, add `A OR B` (negation of `~A AND ~B`) to the KB. Resolving with `~A` gives `B`; resolving with `~B` gives the empty clause ■.

DPLL trace: Unit propagation/branching leads to `B=false`; clause 6 then forces `E=false`; clause 1 forces `A=false`. The remaining `C,D,F` are free. Thus **8 satisfying models over six symbols**. A non-entailed query is `D`, since some models set `D=false`. Its negation is not entailed either.

Comparative interpretation

Truth-table checking uses up to `2^m` valuations; it is exhaustive and easy to audit but poor when `m` is large. Resolution is refutation-complete for propositional CNF, and can generate many redundant resolvents. DPLL uses unit propagation, pure literals and branching and usually avoids exhaustive full valuations. For implication chains of lengths 10,20,40,80, query the final variable given the initial fact; chain-style unit propagation is linear in chain length, while naive truth tables scale exponentially. A tiny two-symbol KB may make brute-force model checking faster than a resolution engine because implementation overhead dominates. Compare **measured** timings, not just asymptotic predictions. For non-entailed queries, a saturation-based resolution prover terminates when no new clauses remain, provided it performs duplicate detection.

Design choice: ask about symbol count, clause width, Horn/definite-clause structure, query count, consistency, repeated queries, and whether an explanation/proof is required. Cache and reuse the KB where possible.

Task 3 — First-order knowledge engineering (7 marks)

Crime domain: vocabulary and seven-step process

Task: determine whether `Criminal(West)`. Relevant entities: West, Nono, America, missile M1. Predicates: `American(x), Missile(x), Weapon(x), Owns(x,y), Sells(x,y,z), Enemy(x,y), Hostile(x), Criminal(x)`. Use predicates rather than functions for many-to-many relations (ownership, sales); a functional `Mother(x)` is reasonable only when unique mothers are assumed.

1. Identify query `Criminal(West)`.

2. Assemble the supplied facts and law.

3. Choose predicates and constants above.

4. Encode:

• `American(x) AND Weapon(y) AND Sells(x,y,z) AND Hostile(z) => Criminal(x)`.

• `Missile(x) => Weapon(x)`.

• `Enemy(x,America) => Hostile(x)`.

• `Missile(x) AND Owns(Nono,x) => Sells(West,x,Nono)`.

5. Facts: `American(West)`, `Enemy(Nono,America)`, `Missile(M1)`, `Owns(Nono,M1)`.

6. Ask `Criminal(West)`; expected **true**.

7. Debug by deleting `Missile(x)=>Weapon(x)`: `Weapon(M1)` and `Criminal(West)` no longer follow from these facts/rules.

The four-variable-free universal rule instances scale as k^3 for the crime rule's three variables. With three one-variable rules, a simplified four-rule grounding count is k^3+3k: k=4:76, k=8:536, k=16:4144, k=32:32864. These are **syntactic ground instances**, not unique useful deductions. Constants and fixed argument positions in some rules can reduce the actually necessary grounding.

Kinship domain: 12 axioms

Assume the predicates `Parent, Sibling, Grandparent, Female, Male, Mother, Father, Child, Ancestor, Aunt, Uncle, Cousin, Daughter` and `Different` (inequality). These are modeling axioms, not claims that every family structure follows a single convention.

1. `Parent(x,y) => Child(y,x)`.
2. `Parent(x,y) AND Female(x) => Mother(x,y)`.
3. `Parent(x,y) AND Male(x) => Father(x,y)`.
4. `Mother(x,y) => Parent(x,y)`.
5. `Father(x,y) => Parent(x,y)`.
6. `Parent(x,y) AND Parent(y,z) => Grandparent(x,z)`.
7. `Parent(x,y) => Ancestor(x,y)`.
8. `Parent(x,y) AND Ancestor(y,z) => Ancestor(x,z)`.
9. `Parent(p,x) AND Parent(p,y) AND x!=y => Sibling(x,y)` (shared-parent definition).
10. `Sibling(x,y) => Sibling(y,x)`.
11. `Sibling(x,p) AND Parent(p,y) AND Female(x) => Aunt(x,y)`.
12. `Sibling(x,p) AND Parent(p,y) AND Male(x) => Uncle(x,y)`.
13. `Parent(p,x) AND Parent(q,y) AND Sibling(p,q) => Cousin(x,y)`.
14. `Parent(p,x) AND Female(x) => Daughter(x,p)`.

Example facts: `Parent(Ana,Ben), Parent(Ben,Cia), Parent(Ana,Dan), Female(Ana), Male(Ben)`; derive `Grandparent(Ana,Cia)`, `Sibling(Ben,Dan)` and `Mother(Ana,Ben)`. Without family facts, open-ended kinship queries cannot be assigned fixed actual truth values.

Task 4 — Unification, lifting and grounding (7 marks)

Important: Standardize variables apart between separate clauses before unifying. Use an occurs check to reject cyclic substitutions.

Hand MGUs (variables lowercase, named constants uppercase)

• `Knows(John,x)` with `Knows(John,Jane)` -> `{x/Jane}`.

• `Knows(John,x)` with `Knows(y,Bill)` -> `{y/John,x/Bill}`.

• `Knows(John,x)` with `Knows(y,Mother(y))` -> `{y/John,x/Mother(John)}`.

• `Knows(John,x)` with `Knows(x,Elizabeth)` -> **standardize second x to x2**: `{x2/John,x/Elizabeth}`. Without standardizing apart, the repeated x may cause an incorrect failure.

• `P(A,B,B)` with `P(x,y,z)` -> `{x/A,y/B,z/B}`.

• `Q(y,G(A,B))` with `Q(G(x,x),y)` -> **no MGU** (after substituting `y=G(x,x)`, equality with `G(A,B)` requires `x=A` and `x=B`, impossible for distinct A,B).

• `Older(Father(y),y)` with `Older(Father(x),John)` -> `{y/John,x/John}`.

• `Knows(Father(y),y)` with `Knows(x,x)` -> **no MGU**, because it would require `y=Father(y)` (occurs check).

If the occurs check is removed, the last pair may incorrectly produce a cyclic binding. Grounding counts for four lifted rules are 76,536,4144,32864 at k=4,8,16,32, versus a fixed lifted rule count of four. Lifted forward chaining unifies only rules with matching facts, rather than materializing every irrelevant ground rule.

Task 5 — First-order resolution (6 marks)

Everyone who loves all animals is loved by someone

Let `Animal(y)` and `Loves(x,y)`. Formalization:

`forall x [(forall y (Animal(y) => Loves(x,y))) => exists z Loves(z,x)]`.

Eliminate implications and move negation inward:

`forall x [exists y (Animal(y) AND ~Loves(x,y)) OR exists z Loves(z,x)]`.

After moving existential quantifiers and Skolemizing with functions of the universal `x`, use `y=f(x)`, `z=g(x)`. CNF:

`Animal(f(x)) OR Loves(g(x),x)`;

`~Loves(x,f(x)) OR Loves(g(x),x)`.

Both Skolem functions depend on `x`; a zero-argument constant would be incorrect for this prenex form.

Jack, dog, Tuna and Curiosity

Facts/rules (a sufficient faithful formalization):

`Dog(D)`, `Owns(Jack,D)`, `Cat(Tuna)`, `Animal(Tuna)`;

`Dog(x)=>Animal(x)`;

`Owns(x,y) AND Dog(y) => Loves(x,z)` for every animal `z` (more precisely `Owns(x,y) AND Dog(y) AND Animal(z) => Loves(x,z)`);

`Loves(x,y) AND Animal(y) => ~Kills(x,y)`;

`Kills(Jack,Tuna) OR Kills(Curiosity,Tuna)`.

CNF rules:

`~Owns(x,y) OR ~Dog(y) OR ~Animal(z) OR Loves(x,z)`;

`~Loves(x,y) OR ~Animal(y) OR ~Kills(x,y)`;

`Kills(Jack,Tuna) OR Kills(Curiosity,Tuna)`.

Proof tree (resolution): instantiate owner rule `{x/Jack,y/D,z/Tuna}` with `Owns(Jack,D)`, `Dog(D)`, `Animal(Tuna)` to derive `Loves(Jack,Tuna)`. Resolve with the no-killing rule `{x/Jack,y/Tuna}` and `Animal(Tuna)` to derive `~Kills(Jack,Tuna)`. Resolve against `Kills(Jack,Tuna) OR Kills(Curiosity,Tuna)` to derive `Kills(Curiosity,Tuna)`. Negate the query (`~Kills(Curiosity,Tuna)`) and resolve to ■. Thus **Curiosity killed Tuna**.

Strategy comparison: breadth-first is fair but may produce many clauses; unit preference resolves short/unit clauses early; set of support prioritizes clauses connected to the negated goal. Measure generated clauses and successful resolution steps using the provided generic resolution engine as a base; first-order resolution additionally requires standardizing apart and unification of complementary literals.

Skolemisation: `exists x P(x)` is satisfiable exactly when `P(c)` (fresh Skolem constant) is satisfiable, but they are not logically equivalent in the expanded signature: a structure may satisfy `exists x P(x)` while `P(c)` is false. On a non-entailed query, fair saturation may halt without an empty clause in a finite case; unrestricted first-order resolution can run indefinitely, so use a step limit and report **unknown**, not false.

Task 6 — Forward vs backward chaining (7 marks)

Crime KB forward trace

Initial: `American(West), Enemy(Nono,America), Missile(M1), Owns(Nono,M1)`.

Iteration 1: `Hostile(Nono), Weapon(M1), Sells(West,M1,Nono)`.

Iteration 2: `Criminal(West)`.

The forward rule order must use **synchronous rounds** to reproduce exactly two iterations; an in-place agenda may derive everything in one pass.

Backward AND/OR proof tree

`Criminal(West)` [OR: choose crime rule]

-> AND `American(West)` [fact], `Weapon(y)` [unify y=M1 via missile rule], `Sells(West,M1,z)` [z=Nono via ownership/sale rule], `Hostile(Nono)` [enemy rule]. All branches succeed. Report goal expansions, unification attempts and maximum depth from instrumented code, since counts depend on evaluation order and indexing.

Ancestor rules and queries

`Ancestor(Mother(x),x)`; `Ancestor(x,y) AND Ancestor(y,z) => Ancestor(x,z)`.

• `Ancestor(Mother(y),John)`: succeeds with `{y/John}`.

• `Ancestor(Mother(Mother(y)),John)`: succeeds with `{y/John}` via two base links and transitivity.

• `Ancestor(Mother(Father(John)),Mother(Mother(John)))`: cannot be established from these rules alone, because the Mother-chain and Father-chain have no connecting axiom. Naive forward chaining nevertheless generates unbounded new terms (`Mother(John)`, `Mother(Mother(John))`, ...) and may never terminate. Remedies: depth-bounded terms, goal-directed backward chaining with memoization, tabling, or a finite-domain restriction. A depth limit yields **unknown** when exhausted, not a proof of falsehood.

For irrelevant fact-base sizes 100,1000,10000, an indexed backward engine should avoid inspecting unrelated predicates; a naive forward scan grows with the fact base. The actual table must be produced by running the code on the specified machine.

Task 7 — Hybrid Wumpus agent design (10 marks)

Stage 1: independent proposals (drafts to be independently reviewed)

Student A proposal: propositional, time-indexed percept and action fluents; CNF and DPLL entailment checks; choose nearest safe unvisited square. Strength: logically complete for finite propositional KB. Weakness: repeated SAT calls and large temporal KB.

Student B proposal: static hazard KB plus current position and visited-state tables; local resolution for safety; frontier queue and shortest safe route. Strength: small incremental KB and explainable decisions. Weakness: unrestricted resolution can be expensive.

Stage 2: comparison

Both use logical safety, percepts and frontier exploration. Retain A's complete SAT entailment and B's static/time-varying separation and efficient routing. Never treat an unproved square as safe.

Stage 3: joint design

Vocabulary: `Pij,Wij,Bij,Sij,Visitedij,Safeij,At(t,i,j),HasGold(t),HasArrow(t)`; percepts and action records indexed by time. **Static KB:** breeze/stench neighbor biconditionals, no hazard at start, exactly one Wumpus. **Dynamic state:** latest position, visited cells, known percepts, gold/arrow status. **Safety check:** `KB |= ~Pij AND ~Wij` by two UNSAT checks (`KB AND Pij`, `KB AND Wij`). Maintain a compact incremental KB; do not re-add unchanged hazard axioms each turn. Retain observation history without materializing every possible time-indexed location fact.

```

KB = static_rules(grid); state = initial_state()

while not terminated:

percept = observe()

KB.add(encode(percept, state.position))

if percept.glitter: GRAB(); state.has_gold = True

frontier = adjacent_to_visited_not_visited()

safe = [q for q in frontier if UNSAT(KB + [P(q)])

and UNSAT(KB + [W(q)])]

if state.has_gold: plan_safe_route_to_start(); continue

if safe: follow_shortest_safe_route_to(safe); continue

if provably_wumpus_in_line and state.has_arrow: SHOOT(); continue
```

if safe_route_to_start: RETURN(); continue

STOP_AND_REPORT_UNDECIDABLE()

```

For each inferred safe cell, retain an explanation (relevant observations, rules, and a refutation trace). If glitter is sensed, grab first; once gold is acquired, prioritize return.

### Stage 4: validation

**Student A theoretical:** propositional DPLL entailment via UNSAT is sound and complete when solver search is exhaustive. Worst-case SAT time is exponential in the number of relevant propositional variables. A naive temporal encoding with T time steps introduces $O(n^2T)$ position fluents, while static hazards use $O(n^2)$ symbols (and exactly-one-Wumpus pairwise encoding adds $O(n^4)$ binary constraints). Restricting to definite Horn clauses allows efficient forward chaining but cannot directly express all negative/disjunctive breeze and exactly-one-Wumpus constraints, reducing expressiveness and possible deductions.

**Student B experimental protocol:** generate at least **30 worlds for each** of 4x4,6x6,8x8 with a fixed seed; sample independent pits at p=0.2 except start and exactly one Wumpus in a non-start cell; place gold on a non-start non-hazard cell. Record `(worlds solved / 30)`, `(unnecessary risk moves)`, `(solver decisions or inference steps / move)`, `(number of KB clauses)`. A conservative agent may return without gold; count this as **not solved**. Use identical world sets for all compared agents. The attached simulation code provides a reproducible starter benchmark, not a fully validated hybrid agent.

**Undecidable safe move example:** start `(1,1)` with a breeze, no stench, and no further observations. Either `(1,2)` or `(2,1)` may contain a pit, and neither is provably pit-free. Logic alone cannot choose a safe forward move.

### Stage 5: cross-defence

Student A should compare empirical inference costs against the worst-case exponential bound; good average performance does not disprove the bound. Student B should note that restricting the KB to local frontier symbols often makes observed costs much smaller than the full $O(n^2)$-variable worst case, while complete SAT still has exponential worst-case behavior.

### Stage 6: evaluation

The Wumpus world is appropriate because it has explicit rules, discrete percepts, and actions that require justified conclusions. Its limits are partial observability, unknown hazards, and the inability of classical entailment to rank risks probabilistically. Before choosing a knowledge-based agent, evaluate expressiveness, data quality, completeness, query latency, explainability, scaling, and what to do when the KB neither entails a proposition nor its negation.

## Contribution record (fill with actual work)

| Activity | Student A evidence | Student B evidence |

|---|---|---|

| Task 1 derivations/checker | | |

| Task 2 resolution/DPLL | | |

| Tasks 3–5 FOL work | | |

| Task 6 chaining | | |

| Task 7 joint design and tests | | |

| Cross-testing and peer explanation | | |

## Measurement sheet (complete after executing notebook/code)

| Experiment | Inputs | Expected | Actual | Steps / models / clauses | Elapsed seconds | Interpretation |

|---|---|---|---|---|---|---|

| Task 1 textbook | 7 symbols | 3 satisfying models | | | | |

| Task 1 n=2 | 8 symbols | 256 candidate models | | | | |

| Task 1 n=3 | 18 symbols | 262144 candidate models | | | | |
```

| Task 2 exercise | 6 symbols | 8 satisfying models | | | | |

| Task 6 crime | given facts | criminal in round 2 | | | | |

| Task 7 4x4 | 30 seeded worlds | measured success | | | | |

| Task 7 6x6 | 30 seeded worlds | measured success | | | | |

| Task 7 8x8 | 30 seeded worlds | measured success | | | | |

Reference: Stuart Russell and Peter Norvig, *Artificial Intelligence: A Modern Approach*, 3rd ed., Chapters 7–9. This document follows the uploaded assignment's task wording and its cited book examples. All extra modeling assumptions are identified in the text.

Executed Experimental Validation

Original eight assignment pages are preserved. Runtime figures are environment-dependent. No measurements are fabricated.

Task 1: Truth-table model checking

not P12: models=128, KB models=3, entailed=True, seconds=6.4e-05.

P22: models=128, KB models=3, entailed=False, seconds=5.6e-05.

not P22: models=128, KB models=3, entailed=False, seconds=5.4e-05.

The 4x4 pit/breeze full model count is 2^32; not enumerated.

Task 2: Propositional resolution

The Exercise 7.20 KB has eight satisfying models over six symbols.

resolution_notA_and_notB: entailed=True, attempts=964, tautologies=248, clauses=77, seconds=0.001097.

resolution_D_not_entailed: entailed=False, attempts=72141, tautologies=38268, clauses=287, seconds=0.085496.

Task 4: UNIFY with occurs check

Test 1: {'?x': 'Jane'}

Test 2: {'?y': 'John', '?x': 'Bill'}

Test 3: {'?y': 'John', '?x': "('Mother', '?y')"}

Test 4: {'?x2': 'John', '?x': 'Elizabeth'}

Test 5: {'?x': 'A', '?y': 'B', '?z': 'B'}

Test 6: None

Test 7: {'?y': 'John', '?x': 'John'}

Test 8: None

Removing the occurs check incorrectly accepts a cyclic substitution: {'?x': "('Father', '?y')", '?y': "('Father', '?y')"}

Ground counts: {'4': 76, '8': 536, '16': 4144, '32': 32864}

Task 5: First-order resolution

bfs: proved=False, status=unknown if limit reached, attempts=1500, generated=830, seconds=0.172457.

unit: proved=True, status=proof found, attempts=1393, generated=768, seconds=0.080846.

sos: proved=True, status=proof found, attempts=78, generated=30, seconds=0.001212.

nonentailed: proved=False, status=saturated, attempts=2, generated=0, seconds=3.5e-05.

A bounded proof search that hits the limit returns unknown, not false.

Task 6: Forward/backward chaining

0 irrelevant facts: forward rounds=2, derived=4, unifications=21, seconds=0.000172; backward goals=9, unifications=9, max depth=8, seconds=5.6e-05.

100 irrelevant facts: forward rounds=2, derived=4, unifications=21, seconds=0.000265; backward goals=9, unifications=9, max depth=8, seconds=5.4e-05.

1000 irrelevant facts: forward rounds=2, derived=4, unifications=21, seconds=0.000741; backward goals=9, unifications=9, max depth=8, seconds=0.000121.

10000 irrelevant facts: forward rounds=2, derived=4, unifications=21, seconds=0.010047; backward goals=9, unifications=9, max depth=8, seconds=0.000774.

Forward trace: iteration 1 derives Hostile(Nono), Weapon(M1), Sells(West,M1,Nono); iteration 2 derives Criminal(West).

Task 7: Seeded 30-world trials per grid

4x4: solved=9/30, fraction=0.3, unnecessary risks=0, frontier assignments per move=20.58, mean percept literals=7.33, seconds=0.0114.

6x6: solved=11/30, fraction=0.367, unnecessary risks=0, frontier assignments per move=205.87, mean percept literals=9, seconds=0.1559.

8x8: solved=6/30, fraction=0.2, unnecessary risks=0, frontier assignments per move=138.99, mean percept literals=15.33, seconds=0.318.

Benchmark uses exact local frontier pit assignments up to 16 frontier squares, exact filtering of possible Wumpus locations and a sound zero-breeze fallback. Gold reached implies a safe route home through visited squares. Shooting and return actions are not simulated.

Remaining limits

This is a comprehensive worked answer and tested implementation appendix, but not every requested algorithmic benchmark is fully executed: generic DPLL tracing, chain-length resolution saturation comparisons, ground-versus-lifted time curves, bounded ancestor queries, and a complete shooting hybrid remain unverified. Student contribution signatures must be completed by the actual pair.

Python source code listings

verified_experiments.py

verified_experiments.py: lines 1–48

```
import itertools,random,time,statistics,json
from collections import deque

def tt(kb, query, symbols):
    t=time.perf_counter(); checked=0; sat=0; counterexample=None
    for vals in itertools.product((False,True),repeat=len(symbols)):
        d=dict(zip(symbols,vals));checked+=1
        if kb(d):
            sat+=1
            if not query(d) and counterexample is None: counterexample=d.copy()
    return dict(models=checked,kb_models=sat,entailed=counterexample is None,
seconds=round(time.perf_counter()-t,6))

def book(d):
    return (not d['P11'] and d['B11']==(d['P12'] or d['P21']) and d['B21']==(d['P11'] or d['P22'] or
d['P31']) and not d['B11'] and d['B21'])
syms=['P11','P12','P21','P22','P31','B11','B21']
results={'task1':{q:tt(book,lambda d,q=q: not d[q] if q!='P22' else d[q],syms) for q in ['P12','P22']}}
results['task1']['entailed_positive_P22']=tt(book,lambda d:d['P22'],syms)
results['task1']['entailed_negative_P22']=tt(book,lambda d:not d['P22'],syms)
results['task1']['three_tests']=[tt(lambda d:d['P'] and (not d['P'] or d['Q']),lambda d:d['Q'],['P',
'Q']),tt(lambda d:d['P'] or d['Q'],lambda d:d['P'],['P','Q']),tt(lambda d:d['P'] and not d['P'],lambda
d:d['Q'],['P','Q'])]
results['task1']['grid_enumeration']={}
for n in [2,3]:
    symbols=[f'P{x}_{y}' for x in range(n) for y in range(n)]+[f'B{x}_{y}' for x in range(n) for y in
range(n)]
    def gridkb(d,n=n):
        if d['P0_0']:return False
        for x in range(n):
            for y in range(n):
                adj=[(a,b) for a,b in ((x+1,y),(x-1,y),(x,y+1),(x,y-1)) if 0<=a<n and 0<=b<n]
                if d[f'B{x}_{y}']!=any(d[f'P{a}_{b}'] for a,b in adj):return False
        return True
    results['task1']['grid_enumeration'][str(n)]=tt(gridkb,lambda d:not d['P0_0'],symbols)
results['task1']['grid_enumeration']['4']={'models_theoretical':2**32,'status':'not executed:
exhaustive enumeration impractical'}

def ex720(d):
    A,B,C,D,E,F=(d[x] for x in 'ABCDEF')
    return (A==(B or E)) and (not E or D) and (not(C and F) or not B) and (not E or B) and (not B or F)
and (not B or C)
results['task2']={q:tt(ex720,lambda d,q=q:not d[q] if q in 'AB' else d[q],list('ABCDEF')) for q in ['A',
'B','D']}
# A complete propositional resolution refutation with instrumentation.
def resolve(clauses):
    t=time.perf_counter(); known=set(map(frozenset,clauses)); attempted=taut=added=0
    while True:
        new=set();ordered=list(known)
        for i,a in enumerate(ordered):
            for b in ordered[i+1:]:
                for lit in a:
                    if -lit in b:
                        attempted+=1;r=(a-{lit})|(b-{-lit})
                        if any(-v in r for v in r):taut+=1;continue
                        r=frozenset(r)
```

verified_experiments.py: lines 49–96

```
                        if not r:return dict(entailed=True,attempted=attempted,tautologies=taut,
clauses=len(known),seconds=round(time.perf_counter()-t,6))
                        if r not in known:new.add(r)
        if not new:return dict(entailed=False,attempted=attempted,tautologies=taut,clauses=len(known),
seconds=round(time.perf_counter()-t,6))
        known|=new
# clause signs: A=1 B=2 C=3 D=4 E=5 F=6
base=[[-1,2,5],[-2,1],[-5,1],[-5,4],[-3,-6,-2],[-5,2],[-2,6],[-2,3]]
results['task2']['resolution_notA_and_notB']=resolve(base+[[1,2]])
results['task2']['resolution_D_not_entailed']=resolve(base+[[-4]])
# Chain family: DPLL-style unit propagation, not full resolution timing.
def unit_chain(length):
    t=time.perf_counter(); facts={0};clauses=[(-i,i+1) for i in range(length)];rounds=0
    for a,b in clauses:
        rounds+=1
        if -a in facts:facts.add(b)
    return dict(length=length,clauses=len(clauses)+1,propagations=rounds,query_true=length in facts,
seconds=round(time.perf_counter()-t,6),full_tt_models=2**(length+1))
results['task2']['chains']=[unit_chain(i) for i in (10,20,40,80)]
# Crime KB forward chaining, synchronous rounds.
def crime(n):
    t=time.perf_counter();facts={('American','West'),('Enemy','Nono','America'),('Missile','M1'),
('Owns','Nono','M1')}
    facts|={('Noise',str(i)) for i in range(n)}
    rounds=[]
    for r in range(2):
        added=set()
        if ('Enemy','Nono','America') in facts:added.add(('Hostile','Nono'))
        if ('Missile','M1') in facts:added.add(('Weapon','M1'))
        if ('Missile','M1') in facts and ('Owns','Nono','M1') in facts:added.add(('Sells','West','M1',
'Nono'))
        if all(v in facts for v in [('American','West'),('Hostile','Nono'),('Weapon','M1'),('Sells',
'West','M1','Nono')]):added.add(('Criminal','West'))
        added-=facts;facts|=added;rounds.append(sorted(str(v) for v in added))
    return dict(irrelevant_facts=n,rounds=rounds,criminal=('Criminal','West') in facts,
seconds=round(time.perf_counter()-t,6))
results['task6']=[crime(n) for n in (0,100,1000,10000)]
# Task 7: intentionally conservative local-sound agent (NOT full SAT hybrid).
def neighbors(cell,n):
    x,y=cell
    return [(a,b) for a,b in ((x-1,y),(x+1,y),(x,y-1),(x,y+1)) if 0<=a<n and 0<=b<n]
def trial(n,seed):
    rng=random.Random(seed); start=(0,0);cells=[(x,y) for x in range(n) for y in range(n) if (x,
y)!=start]
    pits={c for c in cells if rng.random()<.2};wumpus=rng.choice(cells)
    candidates=[c for c in cells if c not in pits and c!=wumpus]
    if not candidates:return dict(solved=False,moves=0,checks=0,kb=0,risk=0)
    gold=rng.choice(candidates); visited={start}; safe={start};
pos=start;checks=0;kb=0;moves=0;has_gold=False
    for _ in range(n*n*3):
        if pos==gold:has_gold=True
        if has_gold:return dict(solved=True,moves=moves,checks=checks,kb=kb,risk=0)
        breezy=any(c in pits for c in neighbors(pos,n));stenchy=wumpus in neighbors(pos,n)
        kb+=2
        if not breezy and not stenchy:
            safe.update(neighbors(pos,n))
        # BFS path through known-safe cells to closest unvisited safe square
```

verified_experiments.py: lines 97–111

```
        q=deque([(pos,[])]);seen={pos};path=None
        while q:
            at,p=q.popleft();checks+=1
            if at in safe and at not in visited:path=p;break
            for nxt in neighbors(at,n):
                if nxt in safe and nxt not in seen:seen.add(nxt);q.append((nxt,p+[nxt]))
        if path is None:break
        pos=path[0];moves+=1;visited.add(pos)
    return dict(solved=False,moves=moves,checks=checks,kb=kb,risk=0)
results['task7']={}
for n in (4,6,8):
    trials=[trial(n,20261113+n*100+i) for i in range(30)]
    results['task7'][str(n)]={'worlds':30,'solved':sum(t['solved'] for t in trials),
'solved_fraction':round(sum(t['solved'] for t in trials)/30,4),'unnecessary_risk_moves':sum(t['risk']
for t in trials),'mean_checks_per_move':round(sum(t['checks'] for t in trials)/max(1,sum(t['moves'] for
t in trials)),2),'mean_kb_observation_literals':round(statistics.mean(t['kb'] for t in trials),2),
'total_moves':sum(t['moves'] for t in trials)}
with open('/mnt/data/ai_assignment/verified_metrics.json','w') as f:json.dump(results,f,indent=2)
print(json.dumps(results,indent=2))
```

full_validation.py

full_validation.py: lines 1–48

```
import time, json, random, statistics, itertools
from collections import deque
OUT={}
def V(x):return isinstance(x,str) and x.startswith('?')
def walk(x,s):
 while V(x) and x in s:x=s[x]
 return x
def occurs(v,x,s):
 x=walk(x,s)
 return v==x or (isinstance(x,tuple) and any(occurs(v,a,s) for a in x[1:]))
def unify(a,b,s=None,check=True):
 if s is None:s={}
 a,b=walk(a,s),walk(b,s)
 if a==b:return s
 if V(a):
  if check and occurs(a,b,s):return None
  return {**s,a:b}
 if V(b):return unify(b,a,s,check)
 if isinstance(a,tuple) and isinstance(b,tuple) and len(a)==len(b) and a[0]==b[0]:
  for x,y in zip(a[1:],b[1:]):
   s=unify(x,y,s,check)
   if s is None:return None
  return s
 return None
def T(f,*a):return (f,*a)
def show(s):return None if s is None else {k:str(walk(v,s)) for k,v in s.items()}
examples=[(T('Knows','John','?x'),T('Knows','John','Jane')),(T('Knows','John','?x'),T('Knows','?y',
'Bill')),(T('Knows','John','?x'),T('Knows','?y',T('Mother','?y'))),(T('Knows','John','?x'),T('Knows',
'?x2','Elizabeth')),(T('P','A','B','B'),T('P','?x','?y','?z')),(T('Q','?y',T('G','A','B')),T('Q',T('G',
'?x','?x'),'?y')),(T('Older',T('Father','?y'),'?y'),T('Older',T('Father','?x'),'John')),(T('Knows',
T('Father','?y'),'?y'),T('Knows','?x','?x'))]
OUT['task4_unification']=[show(unify(a,b)) for a,b in examples]
OUT['task4_occurs_removed']=show(unify(*examples[-1],check=False))
OUT['task4_ground_counts']={str(k):k**3+3*k for k in (4,8,16,32)}
# General definite-clause engines with explicit unification and standardizing apart.
def subst(x,s):
 x=walk(x,s)
 if isinstance(x,tuple):return tuple([x[0]]+[subst(a,s) for a in x[1:]])
 return x
def rename(x,i):
 if V(x):return x+'_'+str(i)
 if isinstance(x,tuple):return tuple([x[0]]+[rename(a,i) for a in x[1:]])
 return x
crime_facts=[T('American','West'),T('Enemy','Nono','America'),T('Missile','M1'),T('Owns','Nono','M1')]
crime_rules=[([T('Enemy','?x','America')],T('Hostile','?x')) ,([T('Missile','?x')],T('Weapon','?x')),
([T('Missile','?x'),T('Owns','Nono','?x')],T('Sells','West','?x','Nono')),([T('American','?x'),
T('Weapon','?y'),T('Sells','?x','?y','?z'),T('Hostile','?z')],T('Criminal','?x'))]
def fc(facts,rules,max_rounds=10):
 facts=set(facts);rounds=[];attempts=0
 for i in range(max_rounds):
  added=set()
  for body,head in rules:
   sols=[{}]
   for atom in body:
```

full_validation.py: lines 49–96

```
    ns=[]
    for s in sols:
     for fact in facts:
      if fact[0]!=atom[0]:continue
      attempts+=1;u=unify(subst(atom,s),fact,s)
      if u is not None:ns.append(u)
    sols=ns
   for s in sols:added.add(subst(head,s))
  added-=facts
  if not added:break
  facts|=added;rounds.append(sorted(map(str,added)))
 return facts,rounds,attempts
def bc(goal,facts,rules,max_depth=20):
 stats={'goals':0,'unifications':0,'max_depth':0,'repeated':0}; seen=set();counter=itertools.count()
 def solve(goals,s,depth):
  if not goals:return s
  if depth>max_depth:return None
  g=subst(goals[0],s);stats['goals']+=1;stats['max_depth']=max(stats['max_depth'],depth)
  key=(str(g),depth)
  if key in seen:stats['repeated']+=1
  seen.add(key)
  for f in facts:
   if f[0]!=g[0]:continue
   stats['unifications']+=1;u=unify(g,f,s)
   if u is not None:
    ans=solve(goals[1:],u,depth+1)
    if ans is not None:return ans
  for body,head in rules:
   if head[0]!=g[0]:continue
   j=next(counter);h=rename(head,j);b=[rename(x,j) for x in body]
   stats['unifications']+=1;u=unify(g,h,s)
   if u is not None:
    ans=solve(b+goals[1:],u,depth+1)
    if ans is not None:return ans
  return None
 ans=solve([goal],{},0)
 return ans is not None,stats
OUT['task6']={}
for n in (0,100,1000,10000):
 facts=crime_facts+[T('Noise',str(i)) for i in range(n)]
 t=time.perf_counter();derived,rounds,tries=fc(facts,crime_rules);tf=time.perf_counter()-t
 t=time.perf_counter();yes,stats=bc(T('Criminal','West'),facts,crime_rules);tb=time.perf_counter()-t
 OUT['task6'][str(n)]={'forward_rounds':len(rounds),'forward_new_facts':sum(map(len,rounds)),
'forward_unifications':tries,'forward_seconds':round(tf,6),'backward_success':yes,'backward':stats,
'backward_seconds':round(tb,6)}
OUT['task6']['round_trace']=fc(crime_facts,crime_rules)[1]
# FOL resolution: standardize apart; factor duplicate literals; exhaustive bounded search.
def lit(sign,atom):return (sign,atom)
def canonical(clause):return tuple(sorted(set(map(str,clause))))
def fol_resolution(clauses,goal,mode='bfs',limit=250):
```

full_validation.py: lines 97–135

```
 start=time.perf_counter();cs=[frozenset(c) for c in clauses]+[frozenset([lit(False,goal)])]
 known={canonical(c) for c in cs};support={len(cs)-1};attempts=0;generated=0;rounds=0
 while attempts<limit:
  pairs=[]
  for i in range(len(cs)):
   for j in range(i+1,len(cs)):
    if mode=='sos' and i not in support and j not in support:continue
    pairs.append((i,j))
  if mode=='unit':pairs.sort(key=lambda p:len(cs[p[0]])+len(cs[p[1]]))
  changed=False
  for i,j in pairs:
   # unique variable names per parent per resolution attempt
   a=[(sign,rename(atom,'a'+str(attempts))) for sign,atom in cs[i]]
   b=[(sign,rename(atom,'b'+str(attempts))) for sign,atom in cs[j]]
   for la in a:
    for lb in b:
     if la[0]==lb[0] or la[1][0]!=lb[1][0]:continue
     attempts+=1
     u=unify(la[1],lb[1])
     if u is None:continue
     r=frozenset((sgn,subst(atom,u)) for sgn,atom in a+b if (sgn,atom)!=la and (sgn,atom)!=lb)
     if not r:return {'proved':True,'attempts':attempts,'generated':generated,
'seconds':round(time.perf_counter()-start,6)}
     key=canonical(r)
     if key not in known:
      known.add(key);cs.append(r);generated+=1;changed=True
      if mode=='sos' and (i in support or j in support):support.add(len(cs)-1)
     if attempts>=limit:break
    if attempts>=limit:break
   if attempts>=limit:break
  rounds+=1
  if not changed:break
 return {'proved':False,'status':'unknown if limit reached' if attempts>=limit else 'saturated',
'attempts':attempts,'generated':generated,'seconds':round(time.perf_counter()-start,6)}
# Jack clauses (dog ownership, animal lover, no killing, disjunction)
d='D';j='Jack';t='Tuna';c='Curiosity'
jack=[[lit(True,T('Dog',d))],[lit(True,T('Owns',j,d))],[lit(True,T('Animal',t))],[lit(False,T('Dog',
'?x')),lit(True,T('Animal','?x'))],[lit(False,T('Owns','?x','?y')),lit(False,T('Dog','?y')),lit(False,
T('Animal','?z')),lit(True,T('Loves','?x','?z'))],[lit(False,T('Loves','?x','?y')),lit(False,T('Animal',
'?y')),lit(False,T('Kills','?x','?y'))],[lit(True,T('Kills',j,t)),lit(True,T('Kills',c,t))]]
OUT['task5']={m:fol_resolution(jack,T('Kills',c,t),m,limit=1500) for m in ('bfs','unit','sos')}
OUT['task5']['nonentailed']=fol_resolution(jack,T('Kills','Curiosity','D'),'sos',limit=150)
with open('/mnt/data/ai_assignment/full_validation_metrics.json','w') as f:json.dump(OUT,f,indent=2)
print(json.dumps(OUT,indent=2))
```

hybrid_benchmark.py

hybrid_benchmark.py: lines 1–48

```
import random,itertools,json,statistics,time
from collections import deque

def adj(c,n):
 x,y=c
 return [(a,b) for a,b in ((x-1,y),(x+1,y),(x,y-1),(x,y+1)) if 0<=a<n and 0<=b<n]
def benchmark(n,seed):
 rng=random.Random(seed);start=(0,0);cells=[(i,j) for i in range(n) for j in range(n)];other=cells[1:]
 pits={c for c in other if rng.random()<.2};w=rng.choice(other)
 golds=[c for c in other if c not in pits and c!=w]
 if not golds:return {'solved':False,'moves':0,'checks':0,'kb':0,'risks':0,'status':'no safe gold
square'}
 gold=rng.choice(golds);pos=start;visited=set();observations={};moves=0;checks=0;kb=0;has_gold=False
 for turn in range(n*n*3):
  visited.add(pos)
  if pos==gold:has_gold=True
  if has_gold:
   # traversed cells are safe, so a return path exists
   return {'solved':True,'moves':moves,'checks':checks,'kb':kb,'risks':0,'status':'gold acquired and
safe return possible'}
  observations[pos]=(any(a in pits for a in adj(pos,n)),w in adj(pos,n));kb=2*len(observations)
  frontier=set(a for v in visited for a in adj(v,n) if a not in visited)
  # Exactly-one Wumpus: filter all candidate cells by stench observations.
  wposs=[c for c in other if c not in visited and all((c in adj(v,n))==s for v,(b,s) in
observations.items())]
  # Pit constraints only involve cells neighboring visited cells. No observed pit-free square can
contain a pit.
  unknown=sorted(frontier);safe_pits=set(unknown)
  # Enumerate all consistent assignments on the frontier. Stop at 2^16 for safety.
  if len(unknown)>16:
   # Sound fallback: only zero-breeze neighbors can be proved pit-free without enumeration.
   safe_pits=set(a for v,(b,s) in observations.items() if not b for a in adj(v,n))
  else:
   anymodel=False
   for bits in itertools.product((False,True),repeat=len(unknown)):
    checks+=1;ps={c for c,bit in zip(unknown,bits) if bit}
    if all(any(a in ps for a in adj(v,n))==b for v,(b,s) in observations.items()):
     anymodel=True;safe_pits-=ps
   if not anymodel:raise RuntimeError('inconsistent pit observations')
  safe={c for c in frontier if c in safe_pits and c not in wposs}
  if not safe:break
  # shortest path from current location via visited safe cells to nearest safe frontier.
  queue=deque([(pos,[])]);seen={pos};path=None
  while queue:
   v,p=queue.popleft()
   if v in safe:path=p;break
   for nxt in adj(v,n):
    if nxt not in seen and (nxt in visited or nxt in safe):seen.add(nxt);queue.append((nxt,p+[nxt]))
  if not path:break
  pos=path[0];moves+=1
  if pos in pits or pos==w:raise RuntimeError('unsound safety proof')
 return {'solved':False,'moves':moves,'checks':checks,'kb':kb,'risks':0,'status':'no provably safe
unvisited cell'}
```

hybrid_benchmark.py: lines 49–54

```
results={}
for n in (4,6,8):
 t=time.perf_counter();trials=[benchmark(n,91001+n*100+i) for i in range(30)]
 results[str(n)]={'worlds':30,'solved':sum(x['solved'] for x in trials),
'solved_fraction':round(sum(x['solved'] for x in trials)/30,3),'unnecessary_risk_moves':sum(x['risks']
for x in trials),'assignment_checks_per_move':round(sum(x['checks'] for x in trials)/max(1,
sum(x['moves'] for x in trials)),2),'mean_observation_literals':round(statistics.mean(x['kb'] for x in
trials),2),'seconds':round(time.perf_counter()-t,4)}
with open('/mnt/data/ai_assignment/hybrid_metrics.json','w') as f:json.dump(results,f,indent=2)
print(json.dumps(results,indent=2))
```